

DOM та Events (Події)

Головна ідея: DOM -- це деревоподібне представлення HTML-документа, з яким JavaScript може взаємодіяти: знаходити елементи, читати/змінювати їх, створювати нові, видаляти. Events -- механізм реакції на дії користувача (клік, введення тексту, відправлення форми). Розуміння того, **як** події рухаються по дереву (bubbling), дозволяє писати ефективний код через делегування.

1. ECMAScript, DOM, BOM

Термін	Значення
ECMAScript (ES)	стандарт, що описує саму мову JavaScript (синтаксис, вбудовані об'єкти) -- ES5, ES6/ES2015 тощо
DOM (Document Object Model)	інтерфейс для роботи з HTML-документом як з деревом об'єктів
BOM (Browser Object Model)	інтерфейс для роботи з самим браузером -- розмір вікна, URL, історія навігації

`document` -- кореневий елемент DOM-дерева, частина глобального об'єкта `window` (як і `console.log` , `alert`).

2. Структура DOM-дерева

Кожен HTML-тег утворює елемент-вузол (element node), кожен фрагмент тексту -- текстовий вузол (text node). Кожен елемент (крім кореневого) вкладений в один батьківський.

Термін	Значення
Батько (parent)	елемент, у який безпосередньо вкладений поточний
Дитина (child)	елемент, безпосередньо вкладений у поточний (перший рівень вкладеності)
Нащадок (descendant)	усі елементи всередині поточного, на будь-якому рівні вкладеності
Сусід (sibling)	елементи зі спільним батьком

3. Пошук елементів

```
element.querySelector(selector) // перший елемент за CSS-селектором, або null
element.querySelectorAll(selector) // усі елементи за селектором (псевдомасив), або []

document.querySelector(".heading"); // від кореня документа
someContainer.querySelectorAll(".item"); // тільки всередині someContainer
```

4. Властивості та атрибути

Читання/зміна властивостей

```
const link = document.querySelector(".link");
link.href; // читання атрибута як властивості об'єкта
link.href = "https://x.com"; // зміна -- одразу відображається в DOM
```

Властивість	Призначення
<code>textContent</code>	текст елемента без HTML-тегів (читання і запис)
<code>innerHTML</code>	вміст елемента як рядок HTML (читання і запис, парситься браузером)
<code>style</code>	вбудовані стилі; властивості в camelCase (<code>backgroundColor</code> , не <code>background-color</code>)

`innerHTML` для додавання варто використовувати лише коли елемент порожній або треба повністю замінити вміст -- запис у `innerHTML` перестворює всіх нащадків заново, що дорожче за точкове додавання.

classList -- робота з CSS-класами

Метод	Дія
<code>classList.contains(name)</code>	чи є клас -- true/false
<code>classList.add(name)</code>	додати клас
<code>classList.remove(name)</code>	видалити клас (без помилки, якщо його немає)
<code>classList.toggle(name)</code>	перемикач: додає, якщо немає; видаляє, якщо є
<code>classList.replace(old, new)</code>	замінити один клас на інший

Робота з атрибутами

```
image.hasAttribute("src"); // true / false
image.getAttribute("alt"); // значення або null
image.setAttribute("alt", "New"); // встановити/змінити
image.removeAttribute("alt"); // видалити
```

Власні data-атрибути

```
<button data-action="save">Save</button>

saveBtn.dataset.action; // "save" -- читання (data- відкидається)
saveBtn.dataset.action = "update"; // зміна
saveBtn.dataset.role = "admin"; // додавання нового data-атрибута
```

5. Створення та видалення елементів

```
const heading = document.createElement("h1"); // створено в пам'яті, ще НЕ в DOM
heading.textContent = "Title";
heading.classList.add("main-title");

parent.append(heading); // додати в кінець дітей parent
parent.prepend(heading); // додати на початок дітей parent
heading.remove(); // видалити з DOM
```

insertAdjacentHTML -- вставка без перестворення

```
element.insertAdjacentHTML(position, htmlString);
```

Позиція	Куди вставляє
"beforebegin"	перед element (потребує, щоб element вже був у DOM)
"afterbegin"	всередині element, перед усіма дітьми
"beforeend"	всередині element, після усіх дітей
"afterend"	після element (потребує, щоб element вже був у DOM)

6. Події: основи

```
element.addEventListener(eventName, handler, options);
element.removeEventListener(eventName, handler, options); // handler має бути ТІЄЮ Ж функцією

button.addEventListener("click", (event) => {
  console.log(event.type); // "click"
  console.log(event.currentTarget); // елемент, на якому висить обробник
});
```

Для `removeEventListener` обов'язково передавати посилання на **ту саму** функцію, що була в `addEventListener` -- тому для обробників, які плануєш видаляти, краще одразу оголошувати іменовану функцію, а не анонімну.

7. Поширення подій: bubbling, capturing, делегування

Подія проходить крізь DOM у три фази: спочатку **capturing** (від кореня документа вниз до елемента, де сталася подія), потім **target** (сам елемент), потім **bubbling** (назад угору до кореня). За замовчуванням обробники спрацьовують на фазі bubbling.

Властивість	Значення
<code>event.target</code>	елемент, на якому подія справді сталася (наприклад, конкретна кнопка) -- не змінюється під час bubbling
<code>event.currentTarget</code>	елемент, на якому зараз висить обробник , що виконується -- змінюється залежно від того, де саме прив'язаний слухач

```
<ul id="list">
  <li class="item">Task 1<button class="delete">X</button></li>
  <li class="item">Task 2<button class="delete">X</button></li>
</ul>

// Замість того щоб вішати обробник на КОЖНУ кнопку .delete --
// вішаємо ОДИН обробник на батьківський
і використовуємо bubbling
document.getElementById("list").addEventListener("click", (event) => {
  if (event.target.classList.contains("delete")) {
    event.target.closest(".item").remove(); // closest -- найближчий предок за селектором
  }
});
```

Event delegation (делегування подій): один обробник на батьківському елементі замість окремого обробника на кожній дитині. Працює завдяки bubbling і дозволяє автоматично "підхоплювати" навіть елементи, додані в DOM пізніше (динамічно), без повторного навішування слухачів.

Зупинка поширення

Метод	Що робить
<code>event.preventDefault()</code>	скасовує стандартну дію браузера (перехід за посиланням, перезавантаження при submit) -- НЕ впливає на bubbling
<code>event.stopPropagation()</code>	зупиняє подальше поширення події вгору (bubbling) -- обробники на батьківських елементах вже не спрацюють

`stopPropagation()` варто використовувати обережно і тільки за необхідності -- він може непередбачувано зламати делегування подій, яке хтось інший налаштував вище по дереву.

8. Події клавіатури

```
document.addEventListener("keydown", (event) => {
  console.log(event.key); // символ з урахуванням мови/Shift, напр. "А"
  console.log(event.code); // фізична клавіша, незалежно від мови, напр. "KeyA"
});
```

На практиці частіше слухають `keydown` (не `keyup`) -- вона спрацьовує раніше, користувач швидше бачить реакцію. Події клавіатури обробляють на `document`, а не на конкретному елементі.

9. Події елементів форми

Подія	Коли відбувається
submit	відправлення форми (кнопка type="submit" або Enter в полі) -- слухати на <code><form></code>
input	щоразу при зміні значення текстового поля -- НЕ чекає втрати фокусу (найчастіша подія для полів)
change	для текстових полів -- тільки після втрати фокусу; для select/checkbox/radio -- одразу при виборі
focus / blur	отримання / втрата фокусу елементом

```
form.addEventListener("submit", (event) => {
  event.preventDefault(); // скасувати перезавантаження сторінки
  const login = event.target.elements.login.value; // доступ до полів за атрибутом name
});
```

10. Швидкий довідник

Метод / властивість	Призначення
querySelector(sel) / querySelectorAll(sel)	знайти перший / усі елементи за CSS-селектором
textContent	текст елемента без тегів
innerHTML	HTML-вміст елемента як рядок
style.propertyName	вбудовані стилі (camelCase)
classList.add/remove/toggle/contains/replace	робота з CSS-класами
getAttribute/setAttribute/hasAttribute/removeAttribute	робота з довільними HTML-атрибутами
dataset.name	читання/зміна <code>data-name</code> атрибутів
createElement(tag)	створити новий елемент у пам'яті (ще не в DOM)
append() / prepend()	додати дитину в кінець / на початок
remove()	видалити елемент з DOM
insertAdjacentHTML(pos, html)	вставити HTML-рядок у задану позицію без перестворення вмісту
addEventListener/removeEventListener	додати/зняти обробник події
event.target / event.currentTarget	де подія сталась / де висить активний обробник
event.preventDefault()	скасувати дію браузера за замовчуванням
event.stopPropagation()	зупинити bubbling вгору по дереву
closest(selector)	знайти найближчого предка (або себе), що відповідає селектору
Event delegation	один обробник на батьку замість багатьох на дітях, через bubbling
keydown / keyup	натискання / відпускання клавіші
submit / input / change	відправлення форми / зміна поля вводу / зміна значення з втратою фокусу
focus / blur	отримання / втрата фокусу елементом